B. M. S. COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to VTU, Belagavi

DEPARTMENT OF MACHINE LEARNING

Program: B.E. In Artificial Intelligence and Machine Learning

Academic Year: 2026 - 27, Session: July 2026 - Dec 2026

Natural Language Processing for LLM (24AM7PCLLM)

Alternative Assessment Tool (AAT) Report

Topic: Named Entity Recognition for Resume Parsing: Pretrained vs Fine-Tuned BERT

Submitted by

Student Name: Team Member 1: Team Member 2:	 Thijesh Praveen Uday Kumar M
USN: Team Member 1: Team Member 2:	 1BM23AI200 1BM23AI203
Semester & Section:	7th Semester, Section C

Valuation Report

(to be filled by faculty)

Score:	
Comments:	
Faculty In-charge:	Upeksha Prasad G
Faculty Signature with date:	

Table of Contents

1. Introduction 4
1.1 Background of Natural Language Processing 4
1.2 Importance of the Selected NLP Topic 4
1.3 Objective of the Study 4
1.4 Scope of the Report 5
2. Selected NLP Topic Overview 6
2.1 Description of the NLP Task 6
2.2 Real-World Use Cases 6
2.3 Relevance to the Syllabus and Industry 7
2.4 Key Challenges in This NLP Task 7
3. Real-Time Application / API Identified 8
3.1 Application or API Name 8
3.2 Purpose and Functionality 8
3.3 Architecture or Platform on Which It Is Built 8
3.4 Input/Output Behavior 9
3.5 Limitations and Practical Considerations 11
4. NLP Model / Architecture Study 12
4.1 Theoretical Foundations of the Model 12
4.2 Algorithm(s) Used 12
4.3 Architecture Diagram / Layer-Wise Design 12
4.4 Data Flow in the Model 13
4.5 Mathematical Formulation 13
4.6 Why This Model Suits the Selected NLP Task 14
5. Dataset and Preprocessing 15
5.1 Dataset Description (Source, Size, Structure) 15
5.2 Data Cleaning Steps 16
5.3 Tokenization, Lemmatization, Stopwords Removal 17
5.4 Train/Test Split Strategy 17
5.5 Feature Engineering or Embedding Approach 17
6. Implementation Methodology 18
6.1 Tools, Libraries, and Environment Used 18
6.2 Model Implementation Steps 18
6.3 API Integration 19
6.4 Pseudocode / Workflow Diagram 19
6.5 Hyperparameters and Training Configuration 20
7. Results and Evaluation 21

7.1 Performance Metrics 21
7.2 Visualization of Results 22
7.3 Qualitative and Quantitative Analysis 24
7.4 Error Analysis 26
7.5 Comparison Between Approaches 27
8. Discussion 28
8.1 Interpretation of Results 28
8.2 Strengths and Weaknesses of the Model 28
8.3 Challenges Encountered During Implementation 28
8.4 Impact of Dataset Quality and Preprocessing 29
8.5 How API or Application Enhances the NLP Task 29
9. Conclusion 30
9.1 Summary of the Work Done 30
9.2 Key Learnings and Insights 30
9.3 Real-World Applicability 30
9.4 Future Improvements or Extensions 30
10. References 32

1. Introduction

1.1 Background of Natural Language Processing

Natural Language Processing (NLP) studies how computers can analyse, understand and generate human language. Early systems relied on hand-written rules and lexicons; statistical NLP then introduced probabilistic sequence models such as Hidden Markov Models and Conditional Random Fields, which learned from annotated corpora [1]. The neural era replaced hand-crafted features with learned representations: recurrent networks and BiLSTM-CRF taggers [12] became the standard for sequence labelling, and the Transformer architecture [2] removed recurrence entirely in favour of self-attention. Pretrained Transformer encoders such as BERT [3] are trained once on large unlabelled corpora and then *fine-tuned* on a specific task with a small labelled dataset. This "pretrain, then fine-tune" paradigm underlies today's large language models (LLMs) and is the central idea examined in this report.

1.2 Importance of the Selected NLP Topic

Recruiters and Applicant Tracking Systems (ATS) receive large volumes of resumes in heterogeneous formats (PDF, DOCX, plain text) with no fixed structure. Before candidates can be searched, filtered or matched against a job description, the unstructured resume text has to be turned into structured fields: name, contact details, education, employers, job titles and skills. Named Entity Recognition (NER) is the NLP task that performs exactly this extraction. Commercial resume-parsing services expose this capability through REST APIs that return structured JSON [18], [19], which shows the practical value of the problem.

An off-the-shelf NER model, however, is trained on news text and only knows generic entity types (person, organisation, location). Resumes use a different vocabulary, a different layout and need domain-specific labels such as DEGREE, DESIGNATION or SKILLS. Measuring how much *fine-tuning* closes this gap is therefore both a practical and an instructive question for a course on NLP for LLMs.

1.3 Objective of the Study

1. Build a resume-parsing system based on NER that converts raw resume text (PDF, DOCX or TXT) into a list of typed entities with character spans and a structured JSON record.
2. Measure the **normal performance** of an existing pretrained NER model, *dslim/bert-base-NER* [5], applied to resumes without any training (the "before" condition).
3. Fine-tune the same checkpoint on an annotated resume dataset and measure its performance on the same test data with the same metric (the "after" condition).
4. Study performance-enhancement techniques (learning rate and epochs, class-weighted loss, oversampling, sliding windows, a regex rule layer and text normalisation) as an ablation.
5. Deploy both models behind a REST API and a web demo that shows the before-versus-after difference live.

1.4 Scope of the Report

The study uses the publicly available DataTurks "Resume Entities for NER" dataset [9] (219 unique English resumes, 10 entity types). One model family is studied in two conditions (pretrained and fine-tuned BERT-base); larger LLMs and layout-aware models are discussed only as future work. All numbers in this report were produced by the project scripts and are stored in the results folder of the code repository; nothing has been estimated or copied from other work. Training was performed on a single laptop GPU (NVIDIA GeForce RTX 4050 Laptop GPU) and inference is evaluated on a laptop CPU, reflecting a realistic small-team deployment.

2. Selected NLP Topic Overview

2.1 Description of the NLP Task

Named Entity Recognition is a *sequence-labelling* task: given a sequence of words $w_1 \ldots w_n$, the model assigns each word a label that marks whether it begins an entity, continues an entity, or lies outside any entity. In this project the input is raw resume text and the output is a list of entities, each with a type, the surface text and a character span (start, end).

Labels follow the **BIO (IOB2) scheme** [10]: B-X marks the first word of an entity of type X, I-X marks a following word of the same entity and O marks all other words. With 10 entity types this gives $2 \times 10 + 1 = 21$ labels. Table 1 lists the entity types used. For example, the header line "Ananya Rao / Data Scientist" is tagged *Ananya/B-NAME Rao/I-NAME Data/B-DESIGNATION Scientist/I-DESIGNATION*.

Table 1: Target entity types (all present in the DataTurks annotations)

Entity type	Meaning	Example (from the data)
NAME	Candidate name	Ashok Kunam
EMAIL	Contact link (in this dataset always an Indeed profile URL)	indeed.com/r/Ashok-Kunam/…
LOCATION	City / region of the candidate	Bengaluru
COLLEGE_NAME	School, college or university	Visvesvaraya Technological University
DEGREE	Degree or qualification	Bachelor of Engineering in Engineering
GRADUATION_YEAR	Year a degree was completed	2017
COMPANIES_WORKED_AT	Employer names	Microsoft
DESIGNATION	Job title	Server Support Engineer
YEARS_OF_EXPERIENCE	Stated total experience	2.5 years
SKILLS	Skill lists or skills sections	Python, SQL, …

2.2 Real-World Use Cases

- **Applicant Tracking Systems and recruitment platforms:** automatic population of candidate profiles from uploaded resumes, so recruiters can search by skill, employer or degree.
- **Candidate–job matching:** extracted skills and experience are matched against job descriptions to rank applicants.
- **HR analytics:** aggregating degrees, employers or skills across thousands of applications.
- **Commercial parsing APIs:** services such as Affinda and Textkernel offer resume parsing as a paid API that returns structured JSON [18], [19].
- **Beyond hiring:** the same NER technique is used for invoices, medical notes, legal contracts and news (people, organisations, places) [1].

2.3 Relevance to the Syllabus and Industry

The project covers core syllabus topics of NLP for LLMs: tokenisation (word-level and WordPiece sub-words), sequence labelling, contextual embeddings, the Transformer encoder and self-attention, transfer learning through pretraining and fine-tuning, and evaluation with precision, recall and F1. It also illustrates the industrial workflow of adapting a public pretrained checkpoint from the Hugging Face hub [6] to a domain-specific task and serving it through an API, which is how most production NLP systems are built today.

2.4 Key Challenges in This NLP Task

- **Domain shift:** the pretrained model was trained on Reuters news (CoNLL-2003 [4]); resumes have short, list-like lines, abbreviations and Indian names and institutions that are rare in news.
- **Label-set mismatch:** CoNLL-2003 has only PER, ORG, LOC and MISC; seven of our ten target types (e.g. DEGREE, SKILLS, DESIGNATION) have no counterpart at all.
- **Ambiguity:** the same string can be a company, a college or a skill ("Oracle" as employer vs. skill); job titles and company names appear side by side on one line.
- **Layout noise:** text extracted from PDFs loses columns, tables and line structure; bullets turn into replacement characters.
- **Small, noisy data:** only 219 annotated resumes are available, with inconsistent annotation (Section 8.4), and some entity types are rare (YEARS_OF_EXPERIENCE: 40 spans in total).
- **Long documents:** resumes average 688 words (maximum 3,684), exceeding BERT's 512 sub-token limit.

3. Real-Time Application / API Identified

3.1 Application or API Name

Resume NER API and demo (developed in this project): a FastAPI service with the endpoint *POST /parse* and a React web interface. The service wraps both models studied in this report, so the same resume can be parsed by the pretrained model and by the fine-tuned model and compared side by side.

For real-world context, Table 2 compares our API with two commercial resume-parsing APIs, described only from their public documentation.

Table 2: Our API compared with commercial resume-parsing APIs (public documentation only)

Aspect	This project	Affinda Resume Parser [18]	Textkernel Tx Platform [19]
Access	Local FastAPI service, POST /parse (multipart file) and POST /parse_text	REST API; v3 upload to /v3/documents with API key, workspace and document-type identifiers	SaaS REST API; Base64-encoded document sent with account ID and service key headers
Output	JSON: name, email, phone, skills, education, companies, designations, entities with spans	Structured JSON for the parsed resume	JSON following a published schema
Notes	Two models selectable (pretrained / fine-tuned); open source; runs on a laptop CPU	Documented rate limit and credit-based usage	Asks the caller to supply the document revision date for correct date interpretation

3.2 Purpose and Functionality

The application accepts a resume file and returns (i) every detected entity with its label and character offsets, and (ii) a structured record that groups entities into the fields an ATS needs. In the web demo the user can upload a file, pick one of the sample resumes, or paste text, and then view the result of the pretrained model, the fine-tuned model, or both side by side. For resumes taken from the held-out test split the gold annotation is shown as a third column together with a per-document F1 score, which makes the effect of fine-tuning directly visible.

3.3 Architecture or Platform on Which It Is Built

Figure 1 shows the processing pipeline, which is shared by the command-line tool, the API and the demo. Text is extracted with *pdfplumber* (PDF) or *python-docx* (DOCX, including tables), normalised without changing character offsets, split into words with a regular expression, converted into WordPiece sub-tokens and processed in overlapping windows of 512 sub-tokens with a stride of 128. The BERT encoder and token-classification head produce a label distribution for every sub-token; predictions are taken from the first sub-token of each word and averaged where windows overlap. Finally the label sequence is repaired to valid BIO, an optional regex layer is applied, and the spans are grouped into JSON. The backend is Python (FastAPI, PyTorch, Hugging Face Transformers [6]); the frontend is React built with Vite and served by the same FastAPI process.

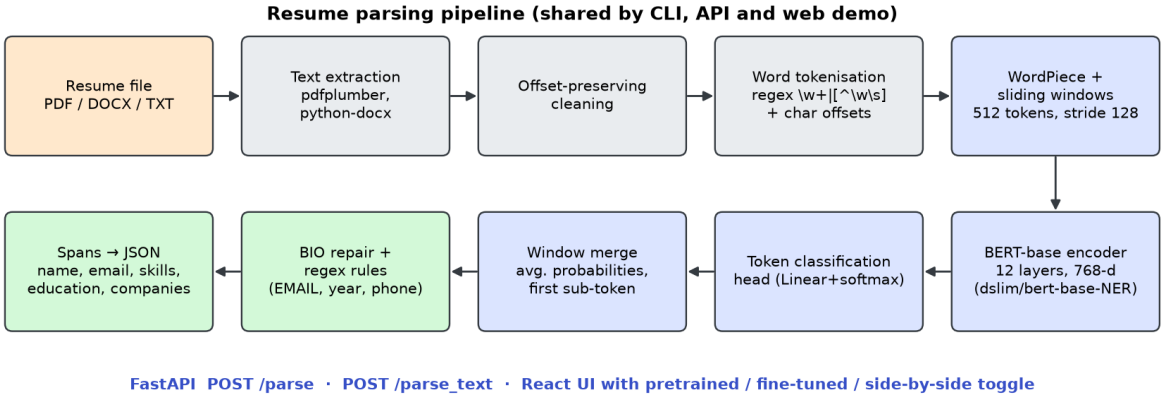


Figure 1: End-to-end architecture of the resume-parsing application

3.4 Input/Output Behavior

Table 3: API input and output

Item	Description
Input	multipart/form-data: file (PDF, DOCX or TXT, ≤ 10 MB), model = pretrained | finetuned | both, rules = true | false
Output fields	name, email, phone, location[], skills[], education{degrees[], colleges[], graduation_years[]}, companies[], designations[], years_of_experience[]
entities[]	list of {text, label, start, end} with character offsets into the extracted text
Diagnostics	model, rules, n_words, latency_ms (model inference only, model load excluded)
Errors	422 for unsupported file types, empty text or image-only PDFs; 413 for files over 10 MB; 503 if the fine-tuned model is missing

Measured on the laptop CPU (14 threads) over the 33 test resumes (average 850 words), inference takes 1.19 s per resume on average for the fine-tuned model (median 0.99 s, 95th percentile 3.01 s) and 1.14 s for the pretrained model. The two models have the same architecture, so the cost is essentially identical; latency grows linearly with the number of 512-token windows. Figure 2 and Figure 3 show the web demo.

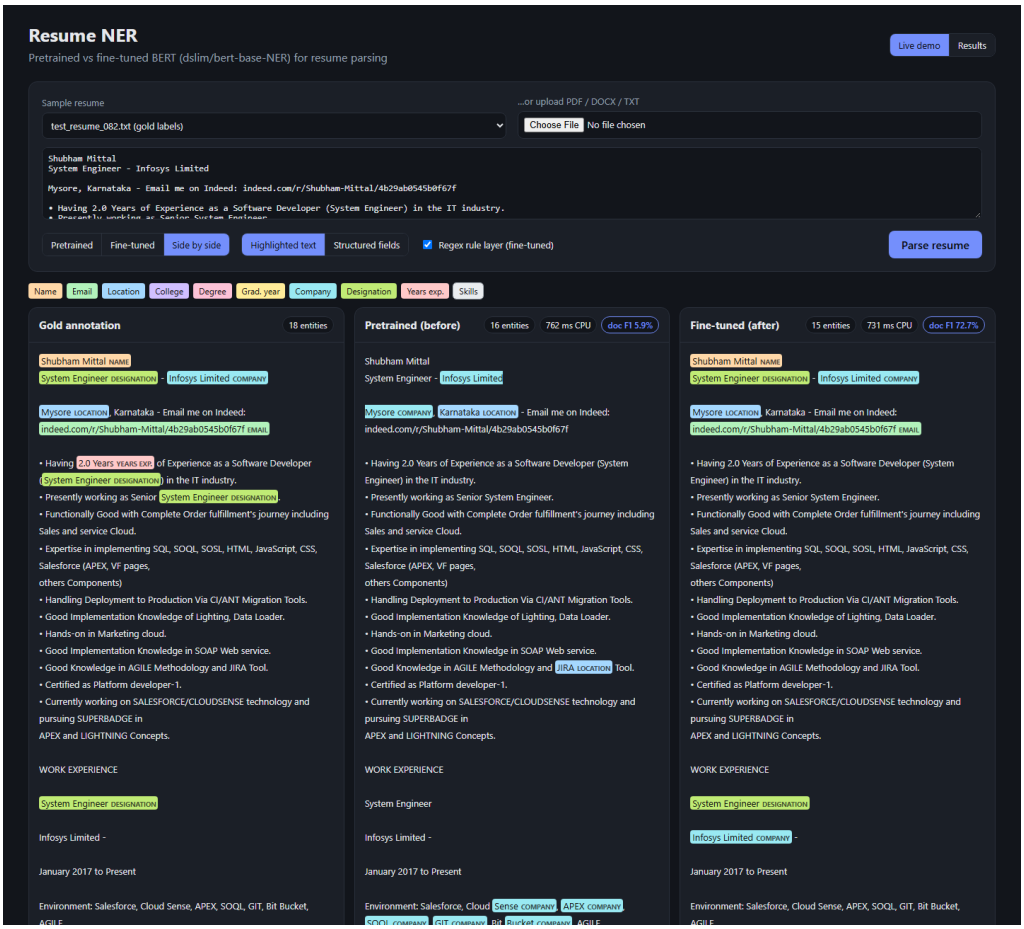


Figure 2: Web demo: the same test resume with gold annotation, pretrained output and fine-tuned output (per-document F1 shown in the panel headers)

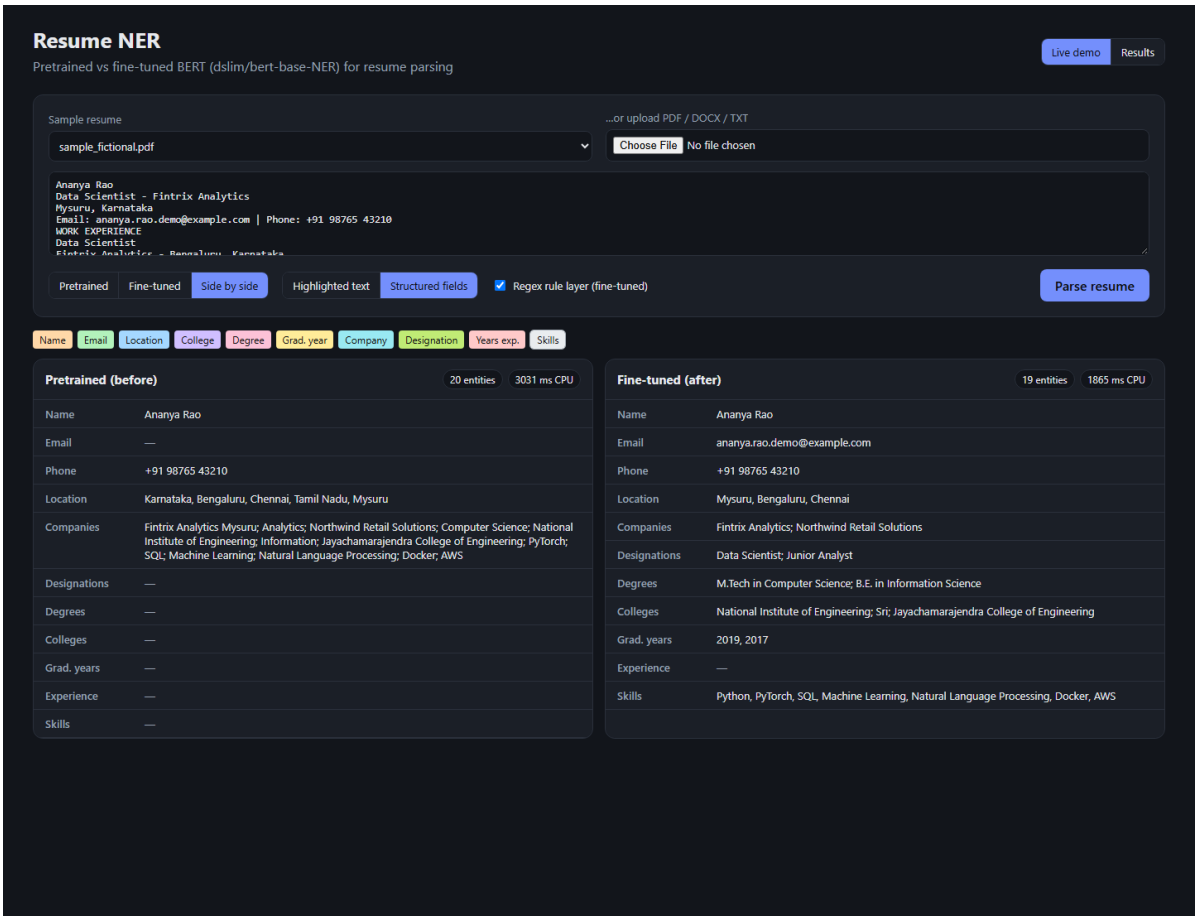


Figure 3: Web demo, structured-field view for a fictional sample resume uploaded as PDF

3.5 Limitations and Practical Considerations

- **Scanned PDFs:** image-only PDFs contain no text layer; the API returns an error because OCR is not part of the pipeline.
- **Multi-column layouts and tables:** pdfplumber reads text line by line, so two-column resumes can interleave the columns; DOCX tables are flattened to "cell | cell" lines.
- **Long resumes:** latency grows with length (95th percentile 3.0 s on CPU); a GPU or a smaller distilled model would be needed for high throughput.
- **Model size:** the fine-tuned model is 411 MB (107.7 M parameters), which must be loaded into memory once at start-up.
- **Accuracy:** with entity-level F1 below 50 % on several types (Section 7), the output should support, not replace, a human recruiter.
- **Privacy and fairness:** resumes contain personal data; a production system must store as little as possible and should be audited for bias before influencing hiring decisions [16].

4. NLP Model / Architecture Study

4.1 Theoretical Foundations of the Model

The Transformer [2] represents every token by a vector and repeatedly updates these vectors with *self-attention*: each token builds a query, compares it with the keys of all other tokens, and takes a weighted average of their values. Because every token can attend to every other token in one step, the model captures long-range context in both directions, which is essential for NER where the type of a word often depends on its neighbours (e.g. "Infosys" after "System Engineer -").

BERT [3] (Bidirectional Encoder Representations from Transformers) is a stack of Transformer encoder layers pretrained on BooksCorpus and English Wikipedia with two self-supervised objectives: masked language modelling (predict randomly masked tokens from both left and right context) and next-sentence prediction. Pretraining gives contextual word representations that already encode syntax and much world knowledge. For a downstream task, a small task-specific layer is added on top and the whole network is *fine-tuned* end to end with a low learning rate.

4.2 Algorithm(s) Used

The model studied is **dslim/bert-base-NER** [5]. According to its model card it is the *bert-base-cased* checkpoint fine-tuned on the English CoNLL-2003 NER dataset [4] (Reuters news) with the labels PER, ORG, LOC and MISC in BIO format, trained on a single NVIDIA V100 GPU with the hyperparameters recommended in the BERT paper. The card reports a test F1 of 91.3 in its evaluation table (the metadata section lists 92.6). It has 107,726,601 parameters.

- **Before (pretrained, normal performance):** the checkpoint is used exactly as published. Its labels are mapped to ours: PER→NAME, ORG→COMPANIES_WORKED_AT, LOC→LOCATION; MISC is discarded.
- **After (fine-tuned):** the same checkpoint is loaded with a new 21-label classification head (the old 9-label head is discarded, option *ignore_mismatched_sizes*), and all 107,735,829 parameters are trained on resumes with cross-entropy loss and the AdamW optimiser [13].

4.3 Architecture Diagram / Layer-Wise Design

Figure 4 shows the layer-wise design. BERT-base has 12 encoder layers, hidden size 768, 12 attention heads of size 64 and a feed-forward inner size of 3072 with GELU activation [20]; each sub-layer is wrapped in a residual connection followed by layer normalisation [21]. The input embedding is the sum of a WordPiece token embedding (cased vocabulary of 28,996 tokens [11]), a learned position embedding (maximum 512 positions) and a segment embedding. The token-classification head is a single linear layer (with dropout) that maps each 768-dimensional hidden state to label scores. The only architectural difference between the two conditions is this head: 9 outputs before, 21 outputs after fine-tuning.

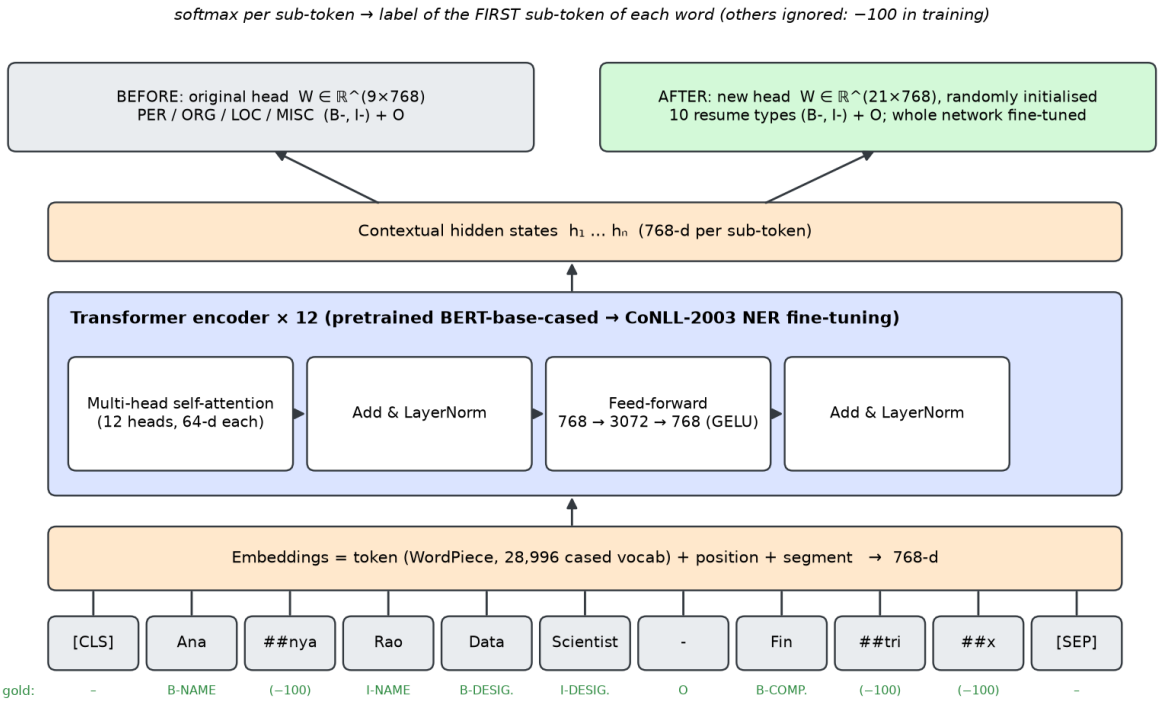


Figure 4: BERT encoder with token-classification head; the real WordPiece segmentation of a resume line is shown at the bottom

4.4 Data Flow in the Model

1. The resume text is split into words with character offsets, and each word is split into WordPiece sub-tokens (e.g. "Fintrix" → *Fin ##tri ##x*).
2. The sub-token sequence is cut into windows of at most 512 tokens ([CLS] … [SEP]) that overlap by 128 tokens, so no word is lost on long resumes.
3. Each window passes through the embedding layer and the 12 encoder layers, producing one contextual vector per sub-token.
4. The classification head and a softmax give a probability distribution over labels for every sub-token.
5. For each word, the distribution of its first sub-token is used; if a word appears in several windows, the distributions are averaged.
6. The arg-max labels are repaired to a valid BIO sequence (an I-X without a preceding X becomes B-X) and decoded into character spans.

4.5 Mathematical Formulation

For a window with hidden matrix H, each attention head computes queries, keys and values Q = HW_Q, K = HW_K, V = HW_V and the scaled dot-product attention [2]:

$$\text{Attention}(Q, K, V) = \text{softmax}(\, Q\, K^{T} / \sqrt{d_k}\,)\, V \qquad (1)$$

where d_k = 64 is the head dimension. The outputs of the 12 heads are concatenated and projected. The classification head turns the final hidden state h_i of sub-token i into a label distribution:

$p(y_i = c \mid x) = \text{softmax}(W h_i + b)_c = \exp(z_c) / \Sigma_c' \exp(z_c'), \quad z = W h_i + b$ (2)

with W ∈ ℝ^(21×768). Training minimises the average cross-entropy over all labelled positions L (the first sub-token of every word; the remaining sub-tokens have label −100 and are ignored):

$\mathcal{L} = -(1/|L|) \Sigma_\{i \in L\} \log p(y_i = g_i \mid x)$ (3)

In the class-weighted ablation each term is multiplied by a weight $\alpha_g = (1/\sqrt{\text{freq}(g)})$ normalised to mean 1. Evaluation uses entity-level precision, recall and F1, where an entity counts as correct only if both its boundaries and its type match exactly (seqeval strict mode [8]):

$P = TP / (TP + FP), \quad R = TP / (TP + FN), \quad F1 = 2PR / (P + R)$ (4)

Micro-averaged scores pool TP, FP and FN over all entity types; macro-averaged scores are the unweighted mean of the per-type scores, so rare types count as much as frequent ones.

4.6 Why This Model Suits the Selected NLP Task

- Bidirectional context: the type of a resume entity depends on both neighbours (a year after a degree is a graduation year; a name next to "Engineer" is a company).
- Already an NER model: dslim/bert-base-NER has learned what names, organisations and places look like, so fine-tuning starts from a strong initialisation rather than from scratch; this matters with only 153 training resumes [3], [14].
- Cased vocabulary: capitalisation is a strong cue for names and organisations in resumes.
- Token-level output fits BIO tagging directly; WordPiece handles unseen names and technical terms by decomposing them into sub-words instead of mapping them to an unknown token.
- Size: at about 108 M parameters, it can be fine-tuned on a 6 GB laptop GPU in minutes and served on a CPU in about 1.2 s per resume.

5. Dataset and Preprocessing

5.1 Dataset Description (Source, Size, Structure)

The dataset is "Resume Entities for NER" published by DataTurks on Kaggle [9] (accessed 16 October 2026 with the Kaggle API; the licence is listed as "unknown" on Kaggle, so the data is used for academic purposes only). It is a single file in JSON-lines format: each of the 220 lines holds one resume as plain text ("content") and a list of annotations, each with a label and character offsets (start, inclusive end). The resumes were taken from Indeed profiles of Indian candidates. Table 4 summarises the data after removing 1 exact duplicate, and Table 5 gives the entity statistics.

Table 4: Dataset summary

Property	Value
Source	Kaggle: dataturks/resume-entities-for-ner [9]
Raw resumes / after de-duplication	220 / 219
Word tokens (regex)	150,753 (mean 688.4, min 75, max 3,684 per resume)
Entity spans (after cleaning)	3,429
Tokens labelled O	86.0 %
Entity types	10 (+ 2 "UNKNOWN" annotations, dropped)

Table 5: Entity statistics per type (all 219 resumes)

Entity type	Spans	Resumes containing it	Mean length (words)	Max length (words)
COMPANIES_WORKED_AT	668	188	1.6	13
DESIGNATION	505	201	2.7	8
SKILLS	455	182	17.0	352
LOCATION	416	198	1.1	3
COLLEGE_NAME	327	193	3.7	10
DEGREE	296	191	3.8	11
EMAIL	250	189	1.3	3
GRADUATION_YEAR	249	104	1.0	2
NAME	223	217	2.1	3
YEARS_OF_EXPERIENCE	40	34	2.0	5

The data is strongly imbalanced: 86.0 % of all words are outside any entity, COMPANIES_WORKED_AT has 668 spans while YEARS_OF_EXPERIENCE has only 40, and SKILLS spans are very long (mean 17.0 words, up to 352) because annotators often marked whole skill sections as one entity. Figure 5 visualises the distribution.

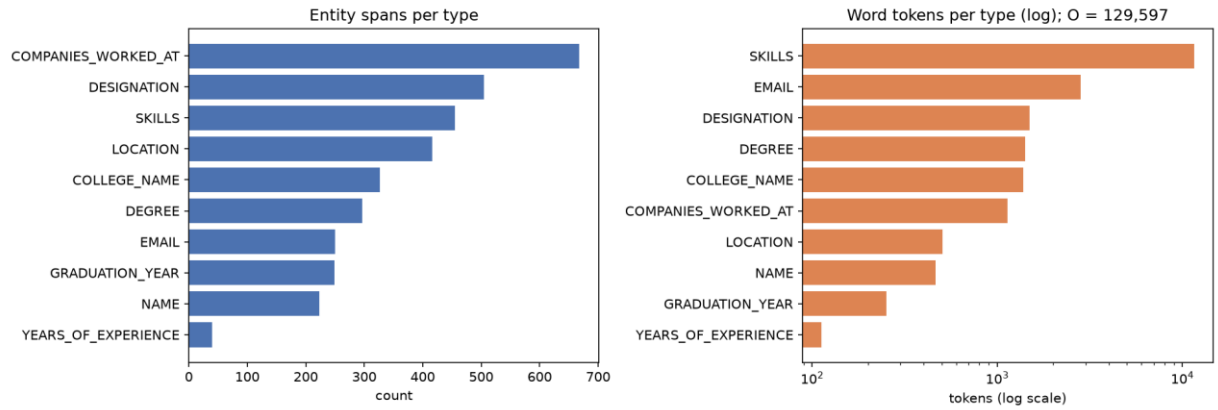


Figure 5: Label distribution: entity spans per type (left) and word tokens per type on a log scale (right)

5.2 Data Cleaning Steps

1. **De-duplication:** resumes were hashed after whitespace normalisation and lower-casing; 1 exact duplicate was removed. No identical text occurs in both training and test sets (checked: 0).
2. **Offset-preserving normalisation:** control characters, Unicode replacement characters (U+FFFD, produced by bullets during PDF conversion), private-use symbols and unusual space characters were replaced by a plain space, one character for one character, so all annotation offsets remain valid.
3. **Span conversion:** DataTurks end offsets are inclusive, so 1 was added to obtain Python-style [start, end) spans. Leading and trailing whitespace was trimmed from every span; empty spans and the "UNKNOWN" label were dropped.
4. **Overlap resolution:** some annotations overlap (e.g. a company inside a designation span). Spans were sorted by start position, longer first, and a span overlapping an already accepted span was dropped.
5. **Word alignment:** spans whose boundaries fall inside a word (the offsets are sometimes shifted by a few characters) were expanded to whole-word boundaries.

Table 6: Span-alignment log produced by data_prep.py

Event	Count
"UNKNOWN" labels dropped	2
Empty spans dropped	1
Overlapping spans dropped	103 (COMPANIES_WORKED_AT 58, LOCATION 13, SKILLS 12, …)
Spans expanded to word boundaries	201
Spans recovered exactly after BIO → span round trip	99.1 %
Original (trimmed) spans recovered character-exactly	94.1 %

As a check, the BIO tags were converted back to spans: 99.1 % of the aligned spans are recovered exactly. The remaining cases are spans whose shifted offsets, after expansion to word boundaries, share a word with a neighbouring entity (e.g. a DESIGNATION span that begins inside the heading word "EXPERIENCE").

5.3 Tokenization, Lemmatization, Stopwords Removal

Two levels of tokenisation are used. First, text is split into **words** with the regular expression \w+|[^\w\s], which keeps alphanumeric runs together and makes every punctuation mark its own token; the character offsets of every word are stored so that predictions can be mapped back to the original text. Second, each word is split into **WordPiece sub-tokens** by the BERT tokenizer [11] (passed with is_split_into_words=True), which preserves the word-to-sub-token alignment.

Stopwords were not removed and no lemmatisation or lower-casing was applied, deliberately. In NER every word must receive a label and entities themselves contain function words ("Bachelor *of* Engineering", "Officer-HR *&* Administration *in* H.& R. Johnson"); removing them would break entity boundaries and destroy the character alignment with the gold spans. Lemmatisation would alter surface forms that the model must copy exactly, and lower-casing would remove capitalisation, one of the strongest cues for names and organisations, which the cased BERT vocabulary relies on.

5.4 Train/Test Split Strategy

The 219 resumes were split at document level (never splitting a resume) into **70 % training, 15 % validation and 15 % test** with a fixed random seed (42). The validation set is used only for model selection (best epoch and hyperparameters); the test set is used once for the final numbers. Because the test set is small, a **5-fold cross-validation** was also run: in each fold one fifth of the resumes forms the test part and the rest is split 85/15 into training and validation. The pretrained model needs no training, so it is simply evaluated on each fold's test part.

Table 7: Resumes and entity spans per split

Split	Resumes	NAME	COMP.	DESIG.	SKILLS	YRS EXP.	All spans
train	153	152	476	351	362	30	2450
val	33	34	75	78	40	3	478
test	33	37	117	76	53	7	501

The CV folds contain 44, 44, 44, 44, 43 test resumes. Note that the test split contains only 7 YEARS_OF_EXPERIENCE spans, so scores for that type are very noisy.

5.5 Feature Engineering or Embedding Approach

No hand-crafted features (TF-IDF, gazetteers, POS tags) are used. Each WordPiece sub-token is represented by BERT's input embedding (token + position + segment, 768 dimensions), and the 12 encoder layers turn these into **contextual embeddings**: the same word "Oracle" receives a different vector in "worked at Oracle" and in "Skills: Oracle, SQL". These contextual vectors are the features consumed by the classification head. In the fine-tuned model the embeddings themselves are also updated, so they adapt to resume vocabulary. Long resumes are handled by the sliding-window scheme of Section 4.4 rather than by truncation (compared in Section 7.3).

6. Implementation Methodology

6.1 Tools, Libraries, and Environment Used

Table 8: Software and hardware environment (from results/environment.txt)

Component	Version / details
Python	3.12.12
PyTorch [17]	2.14.1+cu130
Hugging Face Transformers [6]	5.20.0
seqeval [8]	1.2.2
scikit-learn (splits)	1.9.1
pdfplumber / python-docx	0.11.10 / 1.2.0
FastAPI / Uvicorn	0.143.2 / 0.54.0
Frontend	React 19 + Vite
matplotlib / seaborn	3.11.2 / 0.13.2
Training hardware	NVIDIA GeForce RTX 4050 Laptop GPU (6 GB), CUDA 13.0, bf16 mixed precision
Inference hardware	Laptop CPU (Intel, 14 threads)
Colab	notebooks/train_colab.ipynb runs the identical scripts on a free Colab GPU

6.2 Model Implementation Steps

A. Before: pretrained, normal performance (pretrained_eval.py)

1. Load dslim/bert-base-NER and its tokenizer unchanged; no parameter is updated.
2. Predict word-level CoNLL labels for every resume with the sliding-window inference of Section 4.4.
3. Map labels to the resume label set (PER→NAME, ORG→COMPANIES_WORKED_AT, LOC→LOCATION, MISC→O) and repair the BIO sequence.
4. Evaluate with seqeval in two views: all ten entity types, and only the three overlapping types. An alternative mapping lets an ORG entity count as COLLEGE_NAME when the gold type at its first word is COLLEGE_NAME (an optimistic upper bound).

B. After: fine-tuned performance (finetune.py)

1. Load the same checkpoint with a new 21-label head (id2label/label2id for our labels, ignore_mismatched_sizes=True).
2. Encode each training resume into overlapping 512-token windows; label the first sub-token of each word and set the other sub-tokens and special tokens to −100.
3. Train with AdamW (weight decay 0.01, none on biases and LayerNorm), linear warm-up over 10.0 % of steps followed by linear decay, gradient clipping at 1.0 and bf16 autocast.
4. After every epoch compute the validation loss and validation entity-level F1; keep the weights of the epoch with the best validation micro-F1.

5. Evaluate the selected model on the same test split with the same metric as in A, with and without the regex rule layer.

6.3 API Integration

parse_resume.py implements text extraction and inference once and is imported by the API. Both models are loaded lazily and cached, so model loading happens only on the first request. The FastAPI application exposes POST /parse (file upload), POST /parse_text (JSON), GET /samples, GET /metrics (the before-versus-after summary read from the results files) and serves the built React application and the result figures. Requesting model=both runs the two models on the same text, which is how the demo shows the comparison.

6.4 Pseudocode / Workflow Diagram

Figure 6 shows the experimental workflow; the core training and inference procedures are given below.

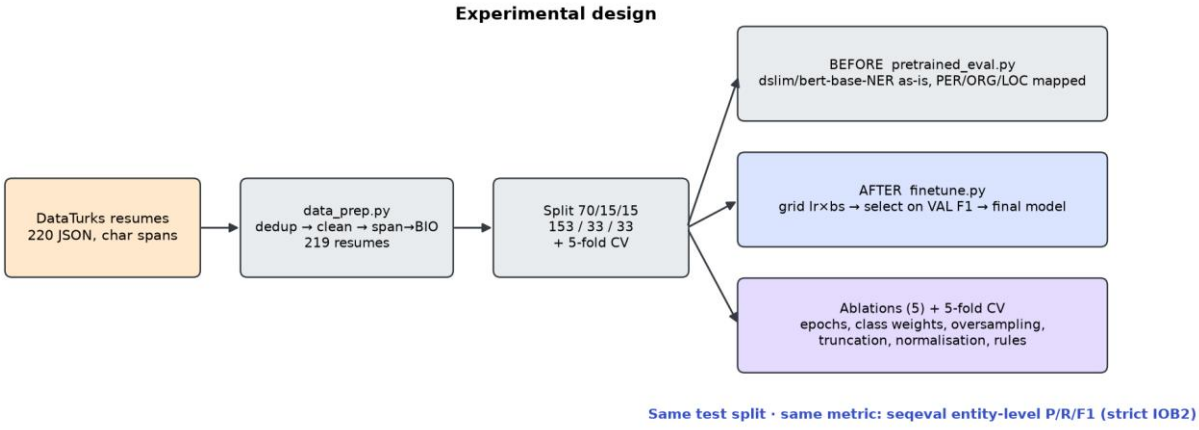


Figure 6: Experimental design: one dataset, one split, two conditions, one metric

```
FINE-TUNE(train, val):
  model ← load("dslim/bert-base-NER", num_labels=21, new head)
  windows ← for each resume: WordPiece(words), cut into 512-token windows, stride 128
  labels  ← first sub-token of word w gets BIO(w); other sub-tokens get −100
  for epoch = 1 … E:
      for batch in shuffle(windows):
          loss ← CrossEntropy(model(batch), labels, ignore=−100)
          AdamW step; linear warm-up/decay schedule; clip grad norm 1.0
      f1 ← seqeval_F1(PREDICT(model, val), gold(val))
      if f1 > best: best ← f1; keep weights
  return model with best validation F1

PREDICT(model, resume):
  words, offsets ← regex_tokenize(clean(resume))
  for each 512-token window: probs ← softmax(model(window))
  P[w] ← mean over windows of probs at the first sub-token of word w
  tags ← repair_BIO(argmax P);  [optional] tags ← regex_rules(tags)
  return spans(tags, offsets)
```

6.5 Hyperparameters and Training Configuration

Learning rate and batch size were selected with a grid search (Table 10) using **validation** micro-F1 only, so the test set did not influence any choice. The selected configuration is lr = 3e-5, batch size 8 (validation F1 42.4 %). Table 9 lists the final configuration.

Table 9: Final training configuration

Hyperparameter	Value
Base checkpoint	dslim/bert-base-NER
Learning rate	3e-5
Batch size	8
Epochs (best epoch kept)	10 (best: 10)
Max length / stride	512 / 128 sub-tokens
Weight decay	0.01
Warm-up	10.0 % of steps, then linear decay
Optimiser	AdamW [13], gradient clipping 1.0
Precision	bf16 autocast
Seed	42
Training windows / resumes	389 / 153
Training time	206 s on NVIDIA GeForce RTX 4050 Laptop GPU

Table 10: Hyperparameter grid (10 epochs each, best epoch selected on validation)

Learning rate	Batch size	Best epoch	Val micro-F1 (%)	Test micro-F1 (%)
2e-5	16	10	27.8	33.3
2e-5	8	8	39.3	42.0
3e-5	16	9	36.0	38.9
3e-5	**8**	**10**	**42.4**	**44.8**
5e-5	16	9	40.3	43.8
5e-5	8	6	42.2	45.9

The test column is reported for transparency only. Smaller batches (more update steps) were clearly better at every learning rate, and the selected configuration is not the one with the highest test score (lr 5e-5 scored marginally higher on test but lower on validation), which illustrates why selection must use the validation set.

7. Results and Evaluation

7.1 Performance Metrics

All scores are entity-level precision, recall and F1 computed with seqeval in strict IOB2 mode [8] on the held-out test split (33 resumes, 501 gold entities). Table 11 gives the per-type comparison of the pretrained ("before") and fine-tuned ("after") model, and Table 12 the overall scores.

Table 11: Per-type results on the test split: before (pretrained, mapped) vs after (fine-tuned). P/R/F1 in %

Entity type	Support	Before P	Before R	Before F1	After P	After R	After F1	Δ F1
NAME	37	23.8	27.0	25.3	87.9	78.4	82.9	+57.5
EMAIL	40	0.0	0.0	0.0	50.0	72.5	59.2	+59.2
LOCATION	54	9.7	61.1	16.7	54.5	66.7	60.0	+43.3
COLLEGE_NAME	49	0.0	0.0	0.0	31.4	44.9	37.0	+37.0
DEGREE	41	0.0	0.0	0.0	42.9	58.5	49.5	+49.5
GRADUATION_YEAR	27	0.0	0.0	0.0	37.2	59.3	45.7	+45.7
COMPANIES_WORKED_AT	117	5.4	49.6	9.7	42.0	58.1	48.7	+39.0
DESIGNATION	76	0.0	0.0	0.0	33.0	43.4	37.5	+37.5
YEARS_OF_EXPERIENCE	7	0.0	0.0	0.0	12.5	14.3	13.3	+13.3
SKILLS	53	0.0	0.0	0.0	8.8	15.1	11.1	+11.1

Table 12: Overall scores on the test split (%)

Metric	Before	After	Δ
Micro F1 — All types	10.3	44.8	+34.5
Macro F1 — All types	5.2	44.5	+39.3
Micro F1 — Overlap types (NAME/COMPANY/LOC)	12.1	56.7	+44.6
Macro F1 — Overlap types (NAME/COMPANY/LOC)	17.2	63.9	+46.6
Micro P / R — all types	6.9 / 20.2	38.7 / 53.1	
Before with ORG→COMPANY or COLLEGE (alt. mapping), micro F1 all types	11.9	—	
After + regex rule layer, micro F1 all types	—	45.6	

Fine-tuning raises micro-F1 over all ten types from 10.3 % to 44.8 % and macro-F1 from 5.2 % to 44.5 %. Even on the three types the pretrained model can express (NAME, COMPANIES_WORKED_AT, LOCATION), micro-F1 rises from 12.1 % to 56.7 %, so the gain is not only due to new labels. The alternative mapping, which lets ORG also count as COLLEGE_NAME, lifts the pretrained score only to 11.9 %.

Table 13: 5-fold cross-validation, mean ± standard deviation over folds (%)

Model	Micro F1	Macro F1
Pretrained (all types)	10.2 ± 1.1	5.9 ± 0.3
Pretrained (overlap types)	12.3 ± 1.2	19.8 ± 0.9
Fine-tuned (all types)	40.7 ± 2.7	42.2 ± 2.1

Model	Micro F1	Macro F1
Fine-tuned + rules (all types)	41.1 ± 2.7	42.4 ± 2.0

Cross-validation confirms the result with lower variance than a single 33-resume test set: the fine-tuned model reaches 40.7 ± 2.7 % micro-F1 against 10.2 ± 1.1 % for the pretrained model; the fine-tuned model is better on every fold (Figure 10). The single test split is somewhat easier than the CV average (44.8 % vs 40.7 %).

Table 14: Efficiency (CPU inference on the test split, fastest of 3 repeats per resume)

Metric	Pretrained	Fine-tuned
Parameters	107,726,601	107,735,829
Mean / median latency per resume (s)	1.14 / 0.96	1.19 / 0.99
95th percentile latency (s)	2.78	3.01
Throughput (words/s)	743	717
Model size on disk	—	411 MB (safetensors)
Training time	none	206 s (GPU)

7.2 Visualization of Results

Figure 7 shows the training of the final model. Training loss falls steadily, while validation loss reaches its minimum at epoch 4 and then rises slightly; validation F1, however, keeps improving until the last epoch (best epoch 10). The model therefore becomes more confident on some wrong predictions (higher loss) while still getting more entities exactly right; selecting the checkpoint by validation F1 rather than loss is the correct choice for an F1-evaluated task.

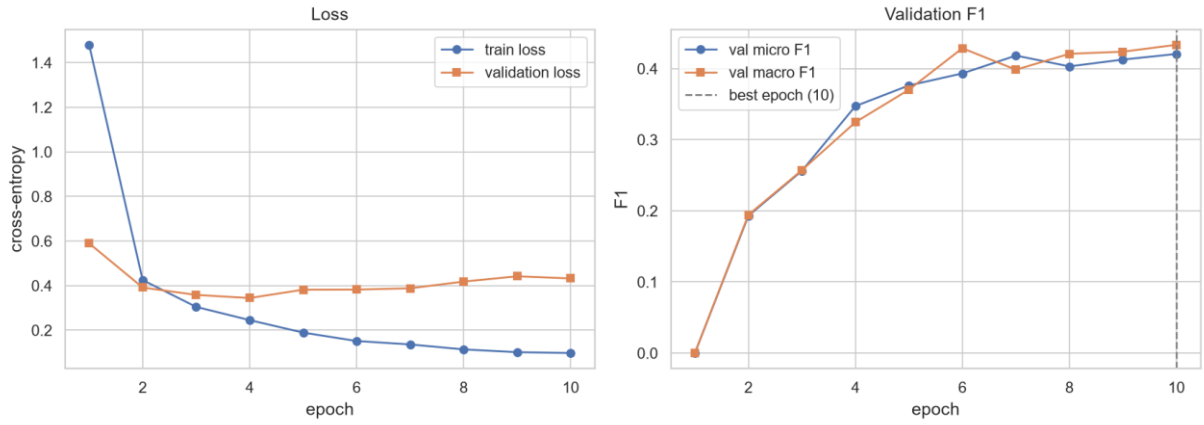


Figure 7: Training and validation loss (left) and validation F1 per epoch (right) for the final model

Figure 8 compares F1 per entity type. The pretrained model scores only on NAME, LOCATION and COMPANIES_WORKED_AT and is zero by construction on the other seven types. Figure 9 shows the word-level confusion matrix of the fine-tuned model (rows normalised).

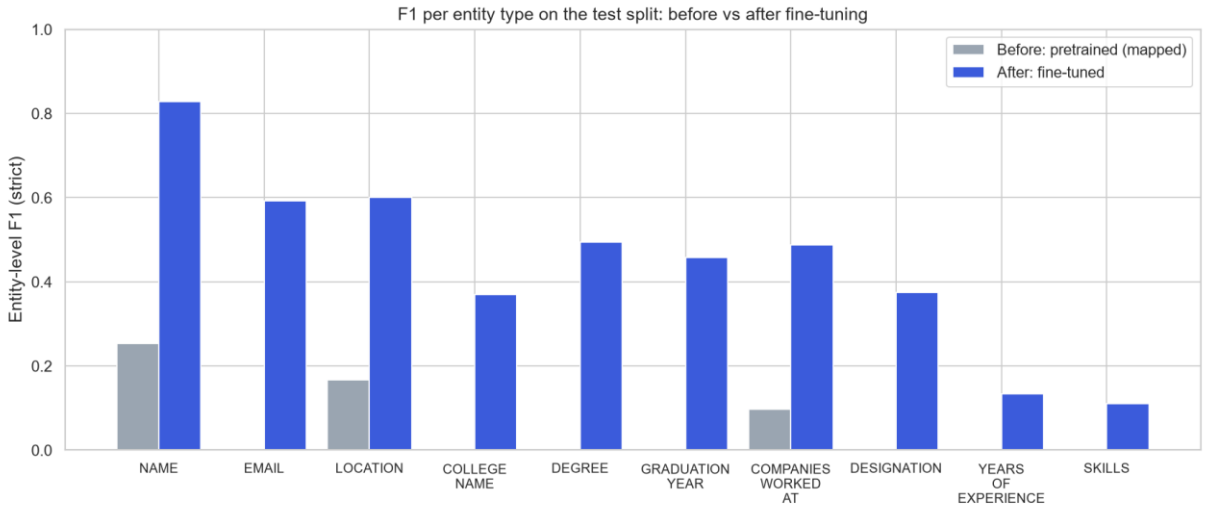


Figure 8: Entity-level F1 per type on the test split, before vs after fine-tuning

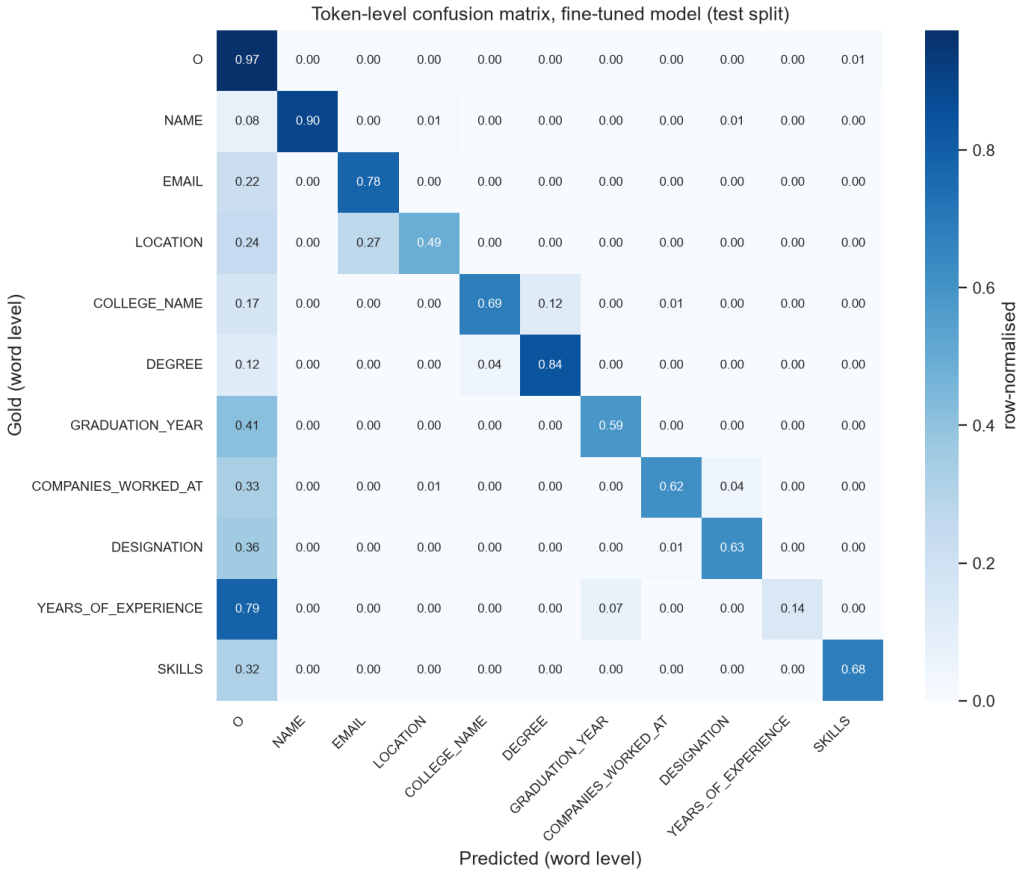


Figure 9: Word-level confusion matrix of the fine-tuned model on the test split (B/I collapsed, row-normalised)

The dominant error is confusion with O (missed or unannotated words), e.g. 78.6 % of YEARS_OF_EXPERIENCE words and 41.4 % of GRADUATION_YEAR words are predicted as O. Between entity types the largest confusions are LOCATION→EMAIL (26.8 %) and COLLEGE_NAME→DEGREE (12.0 %). The LOCATION→EMAIL cell is largely an annotation artefact: in 4 resumes the annotators labelled the Indeed profile URL as LOCATION (Section 8.4).

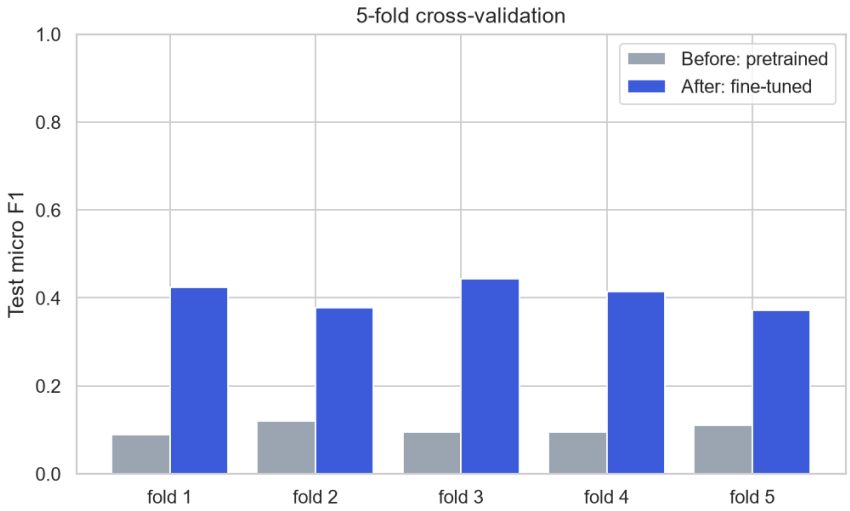


Figure 10: Micro-F1 on each cross-validation fold, before vs after

7.3 Qualitative and Quantitative Analysis

Quantitative: performance-enhancement ablations

Table 15 and Figure 11 report each enhancement as a separate run that changes one setting of the final configuration; all rows use the same test split.

Table 15: Ablation study (%). Each row changes one setting of the final model

Setting	Val micro F1	Test micro F1	Test macro F1
Final fine-tuned (lr=3e-05, bs=8, epochs=10, sliding window)	**42.0**	**44.8**	**44.5**
+ regex rule layer (EMAIL, GRADUATION_YEAR)	—	45.6	45.1
Epochs = 5 (instead of 10)	31.0	31.1	29.8
+ class-weighted loss	35.1	33.9	35.8
+ oversampling rare-entity resumes	44.8	43.2	45.2
Plain truncation (512) instead of sliding window	35.8	38.5	34.4
No text normalisation	42.3	44.7	44.6

- **Learning rate and epochs:** training for 5 instead of 10 epochs drops test micro-F1 to 31.1 %; the model is still under-trained after 5 epochs with this small learning rate. The learning-rate/batch-size grid is in Table 10.
- **Class-weighted loss:** inverse-square-root class weights *hurt* (33.9 % micro, 35.8 % macro). Up-weighting rare I- labels makes the model over-predict long entities and lowers precision.
- **Oversampling** resumes that contain rare types (DEGREE, EMAIL, GRADUATION_YEAR, NAME, YEARS_OF_EXPERIENCE) is roughly neutral: micro-F1 43.2 %, macro-F1 45.2 %.
- **Sliding window vs truncation:** truncating each resume to its first 512 sub-tokens lowers test micro-F1 to 38.5 % and macro-F1 to 34.4 %; entities in the second half of long resumes (education, skills) are lost.

- **Regex rule layer:** adds 0.8 points micro-F1 (45.6 %). The gain comes entirely from EMAIL (59.2 → 65.5 %), where the regex recovers complete Indeed URLs; the graduation-year rule did not change the score.
- **Text normalisation:** removing the offset-preserving clean-up gives 44.7 % vs 44.8 %, i.e. no measurable effect; BERT's tokenizer already ignores most of the noise characters.

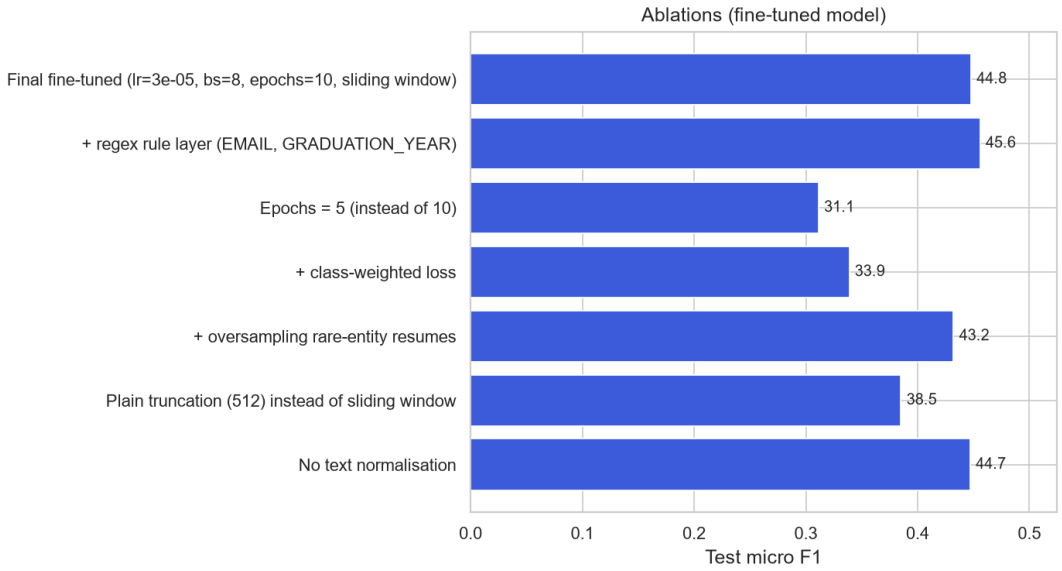


Figure 11: Test micro-F1 of each ablation

Qualitative: the same snippets, both models

Table 16 shows four of the ten test snippets that were analysed (all ten are in results/tables/qualitative.md). The pretrained model labels universities as companies, splits names, and tags state names as locations that the annotators did not mark; the fine-tuned model reproduces the resume-specific labels, including degrees and Indeed profile links.

Table 16: Before vs after predictions on the same test snippets (entity → label)

Snippet (abridged)	Gold	Pretrained (before)	Fine-tuned (after)
Ashok Kunam Team Lead - Microsoft - Email me on Indeed: indeed.com/r/Ashok-Kunam/7aac8767aacf10a0 • Software Engineering professional with over 2.5 years of experience in development,	Ashok Kunam → NAME Team Lead → DESIGNATION Microsoft → COMPANIES_WORKED_AT indeed.com/r/Ashok-Kunam/7aac8767aacf10a0 → EMAIL	Ashok Kunam → NAME Microsoft → COMPANIES_WORKED_AT Kunam → LOCATION Software Engineering → COMPANIES_WORKED_AT	Ashok Kunam → NAME Team Lead → DESIGNATION Microsoft → COMPANIES_WORKED_AT indeed.com/r/Ashok-Kunam/7aac8767aacf10a0 → EMAIL 2.5 years → YEARS_OF_EXPERIENCE
ORACLE SQL - 2015 to 2015 EDUCATION MCA Punjab Technical University - Chandigarh, Chandigarh 2017	MCA → DEGREE Punjab Technical University - Chandigarh, Chandigarh → COLLEGE_NAME BCA → DEGREE Bharati Vidyapeeth University - Pune, Maharashtra → COLLEGE_NAME 2014 → GRADUATION_YEAR Crescent Public School - New Delhi, Delhi → COLLEGE_NAME 2011 → GRADUATION_YEAR	Punjab Technical University → COMPANIES_WORKED_AT Chandigarh → LOCATION Chandigarh → LOCATION BCA Bharati Vidyapeeth University → COMPANIES_WORKED_AT Pune → LOCATION Maharashtra → LOCATION CBSE Crescent Public School → COMPANIES_WORKED_AT New Delhi → LOCATION Delhi → LOCATION	2015 → GRADUATION_YEAR 2015 → GRADUATION_YEAR MCA Punjab Technical University → COLLEGE_NAME 2017 → GRADUATION_YEAR BCA → DEGREE Bharati Vidyapeeth University → COLLEGE_NAME 2014 → GRADUATION_YEAR CBSE Crescent Public School → COLLEGE_NAME 2011 → GRADUATION_YEAR

Snippet (abridged)	Gold	Pretrained (before)	Fine-tuned (after)
Mohini Gupta Server Support Engineer Gurgaon, Haryana - Email me on Indeed: indeed.com/r/Mohini-Gupta/08b5b8e1acd8cf07 Willing to relocate: Anywhere WORK EXPERIENCE Server Support Engineer	Mohini Gupta → NAME Server Support Engineer → DESIGNATION Gurgaon → LOCATION indeed.com/r/Mohini-Gupta/08b5b8e1acd8cf07 → EMAIL Server Support Engineer → DESIGNATION Microsoft → COMPANIES_WORKED_AT	Mohini Gupta → NAME Gurgaon → LOCATION Haryana → LOCATION Microsoft → COMPANIES_WORKED_AT	Mohini Gupta → NAME Server Support Engineer → DESIGNATION Gurgaon → LOCATION indeed.com/r/Mohini-Gupta/08b5b8e1acd8cf07 → EMAIL Server → DESIGNATION Support Engineer → DESIGNATION Microsoft → COMPANIES_WORKED_AT
The divisions were also compared against each other (comparison reports) EDUCATION B.E Visvesvaraya Technological University 2017 Bachelor of Engineering in Engineering	B.E → DEGREE Visvesvaraya Technological University → COLLEGE_NAME Bachelor of Engineering in Engineering → DEGREE	Technological University → COMPANIES_WORKED_AT Engineering → COMPANIES_WORKED_AT Engineering → COMPANIES_WORKED_AT	B.E → DEGREE Visvesvaraya Technological University → COLLEGE_NAME 2017 → GRADUATION_YEAR Bachelor of Engineering in Engineering → DEGREE

7.4 Error Analysis

Every gold entity of the test split was assigned to one error category by comparing it with the fine-tuned predictions (Table 17): *correct* (exact span and type), *boundary* (overlapping prediction of the right type with different boundaries), *type* (exact span, wrong type), *boundary+type*, and *missed* (no overlapping prediction). Predictions that overlap no gold entity are *spurious*. Of 501 gold entities, 266 are correct, 121 have boundary errors, 10 have type errors and 104 are missed; in addition there are 224 spurious predictions.

Table 17: Error categories of the fine-tuned model per entity type (test split, sorted by F1)

Entity type	F1 (%)	Correct	Boundary	Type	Bound.+type	Missed	Spurious
SKILLS	11.1	8	33	0	0	12	20
YEARS_OF_EXPERIENCE	13.3	1	0	1	0	5	7
COLLEGE_NAME	37.0	22	22	2	2	1	16
DESIGNATION	37.5	33	25	0	0	18	38
GRADUATION_YEAR	45.7	16	1	0	0	10	22
COMPANIES_WORKED_AT	48.7	68	14	1	1	33	69
DEGREE	49.5	24	10	0	1	6	13
EMAIL	59.2	29	6	0	0	5	16
LOCATION	60.0	36	4	1	1	12	23
NAME	82.9	29	6	0	0	2	0
Total		**266**	**121**	**5**	**5**	**104**	**224**

- **SKILLS (F1 11.1 %)** is the hardest type. Its errors are mostly *boundary* errors (33 of 53): gold SKILLS spans are whole sections of up to 352 words whose exact start and end are arbitrary (sometimes including the heading "Technical Skills", sometimes not). Under strict matching, predicting the right section with slightly different boundaries counts as both a false positive and a false negative.

- **YEARS_OF_EXPERIENCE (F1 13.3 %)** has only 30 training examples and 7 test examples; it is annotated in only 34 of 219 resumes, although phrases like "2.5 years of experience" occur far more often, so correct predictions are frequently counted as spurious.
- **COLLEGE_NAME and DESIGNATION (≈ 37.0 %)** suffer from boundary disagreements (e.g. whether the city after a university belongs to the name) and from the degree/college and designation/company ambiguity on header lines.
- **Spurious predictions** are often real entities that were never annotated: e.g. "2.5 years" in the summary of resume_009 is predicted as YEARS_OF_EXPERIENCE but is not annotated. 26 resumes contain an Indeed URL that is not annotated as EMAIL, so a correct EMAIL prediction there is counted as an error.

7.5 Comparison Between Approaches

Table 18 summarises the comparison of the two approaches on identical data and metrics.

Table 18: Pretrained vs fine-tuned: summary

Criterion	Pretrained (before)	Fine-tuned (after)
Label set	4 CoNLL types; 3 mappable to resume types	All 10 resume types
Test micro / macro F1 (all types)	10.3 / 5.2 %	44.8 / 44.5 %
Test micro F1 (NAME, COMPANY, LOCATION only)	12.1 %	56.7 %
5-fold CV micro F1	10.2 ± 1.1 %	40.7 ± 2.7 %
NAME F1	25.3 %	82.9 %
Precision / recall (micro, all types)	6.9 / 20.2 %	38.7 / 53.1 %
Training cost	none	206 s on a laptop GPU
CPU latency per resume	1.14 s	1.19 s

The pretrained model's low precision on its own types (5.4 % for COMPANIES_WORKED_AT, 9.7 % for LOCATION) shows that it finds *many* organisations and places, but not the ones a resume parser wants: every university, skill vendor and state name is tagged, while the annotation marks only employers and the candidate's city.

8. Discussion

8.1 Interpretation of Results

Fine-tuning improves micro-F1 by 34.5 points on the test split and by 30.5 points in cross-validation. Two effects explain this. The first is the **label-set shift**: seven of the ten resume types simply do not exist in CoNLL-2003, so the pretrained model scores zero on them by construction. The second is the **domain and annotation-guideline shift**: even on the shared types, the pretrained model's notion of an entity differs from the resume annotation (all organisations vs only employers; any place vs the candidate's location; names split across header lines). Fine-tuning on 153 resumes teaches both the new labels and the resume-specific guidelines, while reusing the language knowledge already in the encoder.

8.2 Strengths and Weaknesses of the Model

- **Strengths:** high accuracy on short, well-defined entities (NAME 82.9 %, LOCATION 60.0 %); robust to unseen names and technical terms thanks to WordPiece; handles arbitrarily long resumes through sliding windows; fast enough for interactive use on a CPU; one model covers all ten types.
- **Weaknesses:** strict-match F1 stays below 40 % for long or ambiguous entities (COLLEGE_NAME, DESIGNATION, YEARS_OF_EXPERIENCE, SKILLS); the model sees text only, not layout; with a validation F1 of 42.0 % and fold-to-fold standard deviation of 2.7 points, results are sensitive to the small amount of data.

8.3 Challenges Encountered During Implementation

- **Span alignment:** DataTurks offsets are inclusive and sometimes shifted by a few characters; spans also overlap. A careful, logged conversion with a round-trip check was required (Table 6).
- **Sub-word alignment:** labels exist per word but BERT works on sub-tokens; only the first sub-token is labelled and predicted, and predictions are merged back to words and characters.
- **Long resumes:** up to 3,684 words; plain truncation loses 6.3 points of micro-F1, so sliding windows with probability averaging were implemented for both training and inference.
- **Tiny data:** 153 training resumes make single-split results noisy; 5-fold CV and validation-only model selection were used to obtain trustworthy numbers.
- **Engineering details:** Windows line endings in exported sample files shifted gold offsets in the demo, and Windows power throttling of background processes inflated CPU latency about five-fold; both were found during testing and fixed or controlled for (benchmarks are run in the foreground, fastest of three repeats).

8.4 Impact of Dataset Quality and Preprocessing

Annotation quality limits the achievable score. Measured on the full dataset: all 250 EMAIL spans are Indeed profile URLs and none is an e-mail address; 26 resumes contain such a URL that was not annotated; in 4 resumes the URL is labelled LOCATION; 31 of 219 resumes with a WORK EXPERIENCE section have no company annotated and 21 of 211 resumes with an EDUCATION section have no college annotated. GRADUATION_YEAR is annotated in only 104 resumes. Such missing labels count correct predictions as false positives and teach the model to stay silent, which lowers both precision and recall.

Preprocessing choices mattered less than expected for text normalisation (no measurable effect) but a great deal for the handling of long documents (sliding windows) and for the label conversion itself: without overlap resolution and word-boundary expansion, 201 spans would have produced invalid or partial BIO sequences.

8.5 How API or Application Enhances the NLP Task

The API turns the model into a usable tool: any resume file is converted to structured JSON in about a second, so it can feed an ATS or a search index. The web demo (Figure 2) makes the effect of fine-tuning tangible: the side-by-side view with gold labels and per-document F1 lets a non-expert see which entities each model finds, and the Results tab presents the measured before-versus-after numbers next to the live predictions. The regex rule layer shows how simple domain knowledge can complement the neural model at the application level.

9. Conclusion

9.1 Summary of the Work Done

A complete resume-parsing system based on NER was built: data preparation with logged span alignment, sliding-window BERT inference, fine-tuning with validation-based model selection, ablations, 5-fold cross-validation, error analysis, a FastAPI service and a React demo. Using the same checkpoint, test split and metric, fine-tuning dslim/bert-base-NER on 153 resumes raised entity-level micro-F1 from 10.3 % to 44.8 % (45.6 % with the regex layer) and macro-F1 from 5.2 % to 44.5 %; cross-validation gives 10.2 % vs 40.7 ± 2.7 %.

9.2 Key Learnings and Insights

- A pretrained NER model does not transfer to a new domain when the label set and annotation guidelines differ; most of its "knowledge" is useful only after fine-tuning re-maps it.
- Fine-tuning is data-efficient: 3 minutes of training on 153 resumes raise micro-F1 from 10.3 % to 44.8 %, because the encoder's language knowledge is reused.
- Evaluation details matter: strict entity matching, a fixed split, validation-only selection and cross-validation change the story considerably compared with token accuracy (the model labels 97.4 % of O words correctly, yet entity F1 is under 50 %).
- Data quality is the ceiling: many remaining "errors" are annotation gaps, and long, loosely defined entities such as SKILLS are hard to score with exact matching.
- Not every popular trick helps: class weighting reduced F1 here, while a simple sliding window and a regex rule did help.

9.3 Real-World Applicability

The fine-tuned model already extracts names, locations, Indeed links, employers and degrees well enough to pre-fill candidate profiles for human review, and the API runs on commodity hardware without a GPU. For production use, it would need more and cleaner training data, an OCR step for scanned resumes, monitoring of extraction quality, data-protection safeguards and a bias audit, since automated screening tools can disadvantage candidates if they are not evaluated carefully [16].

9.4 Future Improvements or Extensions

- **Layout-aware models** such as LayoutLM [15], which combine text with 2-D position, to exploit columns, headings and tables in PDF resumes.
- **Larger and cleaner data:** re-annotating the dataset consistently (real e-mail addresses, all employers, skill items instead of whole sections) and adding more resumes from other countries and industries.
- **LLM-based extraction:** prompting or fine-tuning a generative LLM to output the JSON record directly, compared with the BERT tagger on the same test set for accuracy, cost and latency.

- **Better decoding:** a CRF layer or constrained decoding for valid BIO sequences, and relaxed (partial-match) metrics for long entities such as SKILLS.
- **Efficiency:** distillation or quantisation of the model for faster CPU inference.
- **Fairness and bias:** measuring whether extraction quality differs across name origins, genders or institutions before any use in hiring decisions [16].

10. References

[1] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. draft. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 5998–6008.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[4] E. F. Tjong Kim Sang and F. De Meulder, "Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition," in *Proc. CoNLL-2003*, 2003, pp. 142–147.

[5] dslim, "bert-base-NER," Hugging Face model card. [Online]. Available: https://huggingface.co/dslim/bert-base-NER (accessed Oct. 16, 2026).

[6] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP 2020: System Demonstrations*, 2020, pp. 38–45.

[7] Hugging Face, "Token classification," Transformers documentation. [Online]. Available: https://huggingface.co/docs/transformers/tasks/token_classification (accessed Oct. 16, 2026).

[8] H. Nakayama, "seqeval: A Python framework for sequence labeling evaluation," 2018. [Online]. Available: https://github.com/chakki-works/seqeval

[9] DataTurks, "Resume Entities for NER," Kaggle dataset. [Online]. Available: https://www.kaggle.com/datasets/dataturks/resume-entities-for-ner (accessed Oct. 16, 2026).

[10] L. A. Ramshaw and M. P. Marcus, "Text chunking using transformation-based learning," in *Proc. 3rd Workshop on Very Large Corpora*, 1995, pp. 82–94.

[11] Y. Wu et al., "Google's neural machine translation system: Bridging the gap between human and machine translation," arXiv:1609.08144, 2016.

[12] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer, "Neural architectures for named entity recognition," in *Proc. NAACL-HLT*, 2016, pp. 260–270.

[13] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.

[14] M. Mosbach, M. Andriushchenko, and D. Klakow, "On the stability of fine-tuning BERT: Misconceptions, explanations, and strong baselines," in *Proc. ICLR*, 2021.

[15] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, "LayoutLM: Pre-training of text and layout for document image understanding," in *Proc. ACM SIGKDD*, 2020, pp. 1192–1200.

[16] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, "Mitigating bias in algorithmic hiring: Evaluating claims and practices," in *Proc. ACM Conf. Fairness, Accountability, and Transparency (FAT*)*, 2020, pp. 469–481.

[17] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 8024–8035.

[18] Affinda, "Resume parser: Integration," Affinda documentation. [Online]. Available: https://docs.affinda.com/resumes/integration (accessed Oct. 16, 2026).

[19] Textkernel, "Tx Platform v9: Resume Parser – Getting started," developer documentation. [Online]. Available: https://developer.textkernel.com/tx-platform/v9/resume-parser/overview/getting-started/ (accessed Oct. 16, 2026).

[20] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," arXiv:1606.08415, 2016.

[21] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," arXiv:1607.06450, 2016.

Code, data-processing logs, metrics (the JSON files in results/) and figures are in the project repository; every number in this report is read from those files by report/build_report.js.